
Chapitre 5 : SQL– Le Langage de Définition de Données (LDD)

I. Introduction

Ce chapitre présente les instructions SQL qui constituent l'aspect LDD (langage de définition des données). À cet effet, nous allons voir l'instruction de déclaration d'une table avec ses éventuels index et contraintes.

II. Type de données

Type	Description	Taille
INTEGER	Type des entiers relatifs	4 octets
SMALLINT	Idem.	2 octets
BIGINT	Idem.	8 octets
FLOAT	Flottants simple précision	4 octets
DOUBLE PRECISION	Flottants double précision	8 octets
REAL	Synonyme de FLOAT	4 octets
NUMERIC (M, D)	Numérique avec précision fixe.	M octets
DECIMAL (M, D)	Idem.	M octets
CHAR(M)	Chaînes de longueur fixe	M octets
VARCHAR(M)	Chaînes de longueur variable	L+1 avec $L \leq M$
BIT VARYING	Chaînes d'octets	Longueur de la chaîne.
DATE	Date (jour, mois, an)	env. 4 octets
TIME	Horaire (heure, minutes, secondes)	env. 4 octets
DATETIME	Date et heure	8 octets
YEAR	Année	2 octets

III. LDD: Langage de Définition de Données

Les commandes LDD sont:

- CREATE TABLE : permet la création d'une table.
- ALTER TABLE : permet la modification d'une table déjà créée.
- DROP TABLE : permet la suppression d'une table déjà existante dans la base de données.

1. CREATE DATABASE

La création d'une base de données en SQL est possible en ligne de commande.

- **Syntaxe**

Pour créer une base de données qui sera appelé «ma_base» il suffit d'utiliser la requête suivante:

```
CREATE DATABASE ma_base;
```

- **Base du même nom qui existe déjà**

Avec MySQL, si une base de données porte déjà ce nom, la requête retournera une erreur.

Pour éviter d'avoir cette erreur, il convient d'utiliser la requête suivante pour MySQL:

```
CREATE DATABASE IF NOT EXISTS ma_base;
```

L'option IF NO TEXTISTS permet juste de ne pas retourner d'erreur si une base du même nom existe déjà. La base de données ne sera pas écrasée.

- **Options**

Dans le standard SQL, la commande CREATE DATABASE n'existe normalement pas. En conséquent, il revient de vérifier la documentation des différents SGBD pour vérifier les syntaxes possibles pour définir des options. Ces options permettent selon les cas, de définir les jeux de caractères, le propriétaire de la base ou même les limites de connexion.

2. DROP DATABASE

En SQL, la commande DROP DATABASE permet de supprimer totalement une base de données et tout ce qu'elle contient. Cette commande est à utiliser avec beaucoup d'attention car elle permet de supprimer tout ce qui est inclus dans une base: les tables, les données, les index...

- **Syntaxe**

Pour supprimer la base de données «ma_base», la requête est la suivante:

```
DROP DATABASE ma_base;
```

Remarque: cela va supprimer toutes les tables et toutes les données de cette base. Si vous n'êtes pas sûr de ce que vous faites, n'hésitez pas à effectuer une sauvegarde de la base avant de supprimer.

- **Ne pas afficher d'erreur si la base n'existe pas**

Par défaut, si le nom de base utilisé n'existe pas, la requête retournera une erreur. Pour éviter d'obtenir cette erreur si vous n'êtes pas sûr du nom, il est possible d'utiliser l'option IF EXISTS. La syntaxe sera alors la suivante:

```
DROP DATABASE IF EXISTS
```

```
ma_base;
```

3. CREATE TABLE

Pour pouvoir créer une table dans votre base, il faut que vous ayez reçu le privilège CREATE. La commande CREATE TABLE permet de créer une table en SQL. Un tableau est une entité qui est contenu dans une base de données pour stocker des données ordonnées dans des colonnes.

- **Syntaxe**

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] [nomBase.] nomTable  
(colonne1 type1 [NOT NULL | NULL] [DEFAULT valeur1] [COMMENT 'chaine1']  
[, colonne2 type2 [NOT NULL | NULL] [DEFAULT valeur2] [COMMENT 'chaine2']]  
[CONSTRAINT nomContrainte1 typeContrainte1] ...);
```

- **TEMPORARY** : pour créer une table qui n'existera que durant la session courante (la table sera supprimée à la déconnexion). Deux connexions peuvent ainsi créer deux tables temporaires de même nom sans risquer de conflit. Il faut posséder le privilège CREATETEMPORARY TABLES.
- **IF NOT EXISTS** : permet d'éviter qu'une erreur se produise si la table existe déjà (si c'est le cas, elle n'est aucunement affectée par la tentative de création).
- **nomTable** : mêmes limitations que pour le nom de la base.

colonne i type i : nom d'une colonne (mêmes caractéristiques que pour les noms des tables) et son type (INTEGER, CHAR, DATE...). « ; » : symbole par défaut qui termine une instruction MySQL en mode ligne de commande (en l'absence d'un autre délimiteur).

Pour chaque colonne, il est également possible de définir des options telles que :

-
- **NOT NULL** : empêche d'enregistrer une valeur nulle pour une colonne.
 - **DEFAULT**: attribuer une valeur par défaut si aucune donnée n'est indiquée pour cette colonne lors de l'ajout d'une ligne dans la table.
 - **PRIMARYKEY**: indiquer si cette colonne est considérée comme clé primaire pour un index.

Les contraintes

Les contraintes sont des règles permettant d'assurer une certaine intégrité des données :

- Un attribut (ou un ensemble d'attributs) constitue(nt) la clé de la relation.
- Un attribut doit toujours avoir une valeur. (Contrainte NOT NULL).
- Un attribut dans une table est lié à la clé primaire d'une autre table(*intégrité référentielle*).
- La valeur d'un attribut doit être unique au sein de la relation.

3.1. Clé primaire: PRIMARY KEY

Il peut y avoir plusieurs clés, mais l'une d'entre elles doit être choisie comme clé primaire :

- ✓ Choix capital : la clé primaire est la clé utilisée pour référencer une ligne et une seule à partir d'autres tables.
- ✓ Est spécifiée avec l'option PRIMARY KEY

Exemple1

On souhaite créer une table utilisateur, dans laquelle chaque ligne correspond à un utilisateur inscrit sur un site web. La requête pour créer cette table peut ressembler à ceci:

```
CREATE TABLE utilisateur  
( id INT PRIMARY KEY NOT NULL, nom VARCHAR(100), prenom  
  VARCHAR(100), email VARCHAR(255), date_naissance DATE,  
  pays VARCHAR(255),ville VARCHAR(255), code_postal  
  VARCHAR(5), nombre_achat INT);
```

Voici des explications sur les colonnes créées :

- **id**: identifiant unique qui est utilisé comme clé primaire et qui n'est pas nulle.

-
- **nom:** nom de l'utilisateur dans une colonne de type VARCHAR avec un maximum de 100 caractères au maximum.
 - **prenom:** idem mais pour le prénom.
 - **email :** adresse email enregistré sous 255 caractères au maximum.
 - **date_naissance:** date de naissance enregistré au format AAAA-MM-JJ(exemple:1973-11-17).
 - **pays :** nom du pays de l'utilisateur sous 255 caractères au maximum.
 - **ville:** idem pour la ville.
 - **code_postal :** 5 caractères du code postal.
 - **nombre_achat:** nombre d'achat de cet utilisateur sur le site.

Exemple2

```
CREATE TABLE Internaute (email VARCHAR (50) NOT NULL, nom
                           VARCHAR (20) NOT NULL, prenom VARCHAR (20),
                           Mot De Passe VARCHAR (60) NOT NULL, annéeNaiss
                           DECIMAL (4),PRIMARY KEY (email));
```

- Clé constituée de plusieurs attributs :

```
CREATE TABLE Notation (idFilm INTEGER NOT NULL, email
                        VARCHAR(50) NOT NULL, note INTEGER DEFAULT0,
                        PRIMARYKEY(titre,email));
```

3.2. Clé secondaire: *UNIQUE*

On spécifie que la valeur d'un attribut est unique pour l'ensemble de la colonne.

```
CREATE TABLE Artiste (idINTEGER NOT NULL, nom VARCHAR (30) NOT
                        NULL, prenom VARCHAR (30) NOTNULL, anneeNaiss INTEGER, PRIMARY
                        KEY (ID), UNIQUE(nom, prenom));
```

3.3. Clé étrangère: *FOREIGNKEY*

Ce sont des attributs qui font référence à une ligne dans une autre table.

```
CREATE TABLE Film (idFilm INTEGER NOTNULL,
                    nom VARCHAR (50) NOTNULL, année
                    INTEGER NOT NULL,
                    idMES INTEGER, codePays INTEGER, PRIMARYKEY
```

```
(idFilm),FOREIGN KEY (idMES)
REFERENCES Artiste(idMES), FOREIGN KEY (codePays)
REFERENCES Pays (codePays) );
```

FOREIGNKEY : Référence la clé primaire de la table *Artiste*.

Le SGBD vérifiera, pour toute modification pouvant affecter le lien entre les deux tables, que la valeur d'idMES correspond bien à une ligne d'Artiste.

Modifications contrôlées

1. l'insertion dans *Film* avec une valeur inconnue pour *idMES*
2. la destruction d'un artiste
3. la modification d'id dans *Artiste* ou de *idMES* dans *Film*.

Remarque : Si une violation d'une contrainte d'intégrité est détectée, par défaut, la mise à jour est rejetée. On peut demander la répercussion de cette mise à jour de manière à ce que la contrainte soit respectée grâce à UPDATE et ON DELETE.

```
CREATE TABLE Film (idFilm VARCHAR NOT NULL, année INTEGER NOT
NULL, idMES INTEGER, codePays INTEGER, PRIMARYKEY
(idFilm),FOREIGN KEY (idMES) REFERENCES Artiste(idMES),ON
DELETE SET NULL,FOREIGN KEY (codePays) REFERENCES Pays
(codePays) );
```

La destruction d'un metteur en scène déclenche la mise à NULL de la clé étrangère idMES pour tous les films qu'il a réalisés.

3.4. Énumération des valeurs possibles: CHECK

```
CREATE TABLE Film (idFilm VARCHAR NOT NULL,
Année INTEGER CHECK (année BETWEEN 1890 AND 2000) NOT NULL,
genre VARCHAR (10) CHECK (genre IN ('Histoire', 'Western', 'Drame')), idMES
INTEGER, codePays INTEGER, PRIMARY KEY (idFilm),
FOREIGNKEY (idMES) REFERENCES Artiste(idMES), ON
DELETE SET NULL, FOREIGNKEY (codePays)
REFERENCES Pays (codePays));
```

4. ALTER TABLE

La commande ALTER TABLE en SQL permet de modifier une table existante. Il est ainsi possible d'ajouter une colonne, d'en supprimer une ou de modifier une colonne existante, par exemple pour changer le type.

D'une manière générale, la commande s'utilise de la manière suivante:

```
ALTER TABLE nom_table  
Instruction ;
```

Le mot-clé «instruction» ici sert à désigner une commande supplémentaire, qui sera détaillée ci-dessous selon l'action que l'on souhaite effectuer: ajouter, supprimer ou modifier une colonne.

4.1. Ajouter une colonne

L'ajout d'une colonne dans une table est relativement simple et peut s'effectuer à l'aide d'une requête ressemblant à ceci:

```
ALTER TABLE nom_table  
ADD nom_colonne type_donnees ;
```

Exemple

Pour ajouter une colonne qui correspond à une rue sur une table utilisateur, il est possible d'utiliser la requête suivante:

```
ALTER TABLE utilisateur  
ADD adresse_rue VARCHAR(255) ;
```

4.2. Supprimer une colonne

Une syntaxe permet également de supprimer une colonne pour une table. Il y a deux manières totalement équivalentes pour supprimer une colonne:

```
ALTER TABLE nom_table  
DROP nom_colonne ;
```

Ou (le résultat sera le même)

```
ALTER TABLE nom_table  
DROP COLUMN nom_colonne ;
```

4.3. Modifier une colonne

Pour modifier une colonne, comme par exemple changer le type d'une colonne, il suffit de suivre cette syntaxe :

```
ALTER TABLE nom_table
```

```
MODIFY nom_colonne type_donnees ;
```

Ici, le mot-clé «type_donnees» est à remplacer par un type de données tel que INT, VARCHAR, TEXT, DATE ...

4.4. Renommer une colonne

Pour renommer une colonne, il convient d'indiquer l'ancien nom de la colonne et le nouveau nom de celle-ci.

Pour MySQL, il faut également indiquer le type de la colonne.

```
ALTER TABLE nom_table ;
```

```
CHANGE colonne_ancien_nom colonne_nouveau_nom type_donnees ;
```

Ici «type_donnees» peut correspondre par exemple à INT, VARCHAR, TEXT, DATE ...

5. DROPTABLE

La commande DROP TABLE en SQL permet de supprimer définitivement une table d'une base de données. Cela supprime en même temps les éventuels index, trigger, contraintes et permissions associées à cette table.

Remarque: il faut utiliser cette commande avec attention car une fois supprimée, les données sont perdues. Avant de l'utiliser sur une base importante il peut être judicieux d'effectuer un backup (une sauvegarde) pour éviter les mauvaises surprises.

- **Syntaxe**

Pour supprimer une table «nom_table», il suffit simplement d'utiliser la syntaxe suivante :

```
DROP TABLE nom_table ;
```

A savoir: s'il y a une dépendance avec une autre table, il est recommandé de les supprimer avant de supprimer la table. C'est le cas par exemple s'il y a des clés étrangères.

Intérêts

Il arrive qu'une table soit créée temporairement pour stocker des données qui n'ont pas vocation à être réutiliser. La suppression d'une table non utilisée est avantageuse sur plusieurs aspects :

- **Éviter des erreurs** dans le futur si une table porte un nom similaire ou qui porte à confusion.
- **Libérer de la mémoire** et alléger le poids des backups
- Lorsqu'un développeur ou administrateur de base de données découvre une application, il est **plus rapide de comprendre le système** s'il n'y a que les tables utilisées qui sont présentes.

Exemple de requête

Imaginons qu'une base de données possède une table "client_2009" qui ne sera plus jamais utilisé et qui existe déjà dans un ancien backup. Pour supprimer cette table, il suffit d'effectuer la requête suivante:

```
DROP TABLE client_2009 ;
```

L'exécution de cette requête va permettre de supprimer la table.